

Q1. The spam detection API was containerized using both a naive single-stage Docker build and a multi-stage Docker build. The naive image, `spam-api:naive`, uses `python:3.14-bookworm`, installs all dependencies, copies the training and application files, generates the dataset, and trains the model inside the final image.

The multi-stage image, `spam-api:multistage`, performs the training in a builder stage and uses `python:3.14-slim-bookworm` for the final runtime image. Only the trained `model.joblib` and application code are copied to the runtime stage.

Both images were tested successfully. The `/healthz` endpoint returned `{"status":"ok"}`, and `/predict` returned `{"label":"spam"}` for “WIN a FREE iPhone now!”. The measured content size was 494,193,934 bytes (about 494 MB) for the naive image and 140,547,921 bytes (about 141 MB) for the multi-stage image, giving a 71.56% reduction. This reduction is mainly because the multi-stage image uses a smaller runtime base and does not include the training dependencies, training code, dataset generation files, and builder environment.

Q2. The spam detection API was extended to use Redis for caching predictions. The API first checks Redis using the exact input text as the key. On a cache miss, it performs the prediction, stores the result with a TTL, and returns the label. On a cache hit, the stored label is returned without recomputing the prediction.

Docker Compose was used with two services: the API and the official `redis:7-alpine` image. The API successfully connected to Redis using the Compose service name. For two identical requests, the first request took 0.006663 s, while the second took 0.001724 s, which is about a 74.1% reduction in response time.

Docker Compose is mainly used to run and connect multiple containers on a single machine, whereas Kubernetes provides cluster-level orchestration, scheduling, self-healing, scaling, and rolling updates.

Q3. An Indexed Kubernetes Job was used to validate eight CSV shards in parallel. The Minikube cluster had two nodes, each with 2 allocatable CPUs, giving 4 CPUs in total. The Job was configured with 8 completions and a parallelism of 4. Each validator requested 550m CPU, so four validator Pods could run concurrently within the available CPU resources.

The Job used `completionMode: Indexed`, where each Pod received an index through `JOB_COMPLETION_INDEX` and processed the corresponding shard. The Pod and node names were obtained using the Kubernetes Downward API. The eight shards contained 20 records each, with invalid-record counts from 1 to 8.

The validator image was built as `signup-validator:v1` and loaded into Minikube. During execution, four Pods were simultaneously in the Running state, showing that the configured parallelism was reached. The Job completed successfully with 8/8 completions.

The results were collected through the Kubernetes API by reading the logs of the completed Pods instead of using a shared volume. This avoids depending on node-local storage when Pods run on different nodes. The eight shards had 1, 2, 3, 4, 5, 6, 7, and 8 invalid records respectively, giving a total of 36 invalid records.

If the cluster had three nodes with 2 allocatable CPUs each, there would be 6 CPUs available, so the parallelism could be increased to 6, assuming the same CPU request per Pod.

Q4. The spam detection API was deployed using a Kubernetes Deployment with 2 replicas. The Deployment specifies CPU requests and limits and uses a readiness probe on `/healthz`. A NodePort Service was used to expose the API.

For self-healing, one running Pod was manually deleted. The ReplicaSet managed by the Deployment detected that the number of replicas had fallen below the desired value and created a replacement Pod. The replacement became Ready and restored the Deployment to 2 running replicas.

For the rolling update, the API version was changed from `v1` to `v2`. A new image, `spam-api:v2`, was built and loaded into Minikube, and the Deployment image was updated. The rollout completed successfully using `kubectl rollout status`, and `kubectl rollout history` showed revisions 1 and 2. The final `/healthz` response confirmed that version `v2` was running.

A Deployment is designed for long-running services because it maintains the required number of replicas, replaces failed Pods, and supports rolling updates. A Job is designed for batch processing, where Pods perform a finite task and the workload is considered complete after the required number of completions.